

Organisation. Ce TP s'étale sur deux séances (environ 4 h), voire un peu plus.

- La **partie I** ne parle que de probabilités : expériences aléatoires, événements, fréquences. Pas encore de variables aléatoires.
- La **partie II** simule les variables aléatoires discrètes du cours : Bernoulli, binomiale, géométrique, Poisson, lois finies quelconques... et se termine par un problème.
- Les questions étoilées sont facultatives : pour les élèves les plus à l'aise, à garder pour la fin, ou pour les révisions.

Symboles usuellement utilisés :

(♠) : directement exigible au programme (je ne le mets presque jamais en maths... mais en info c'est bon à savoir)

(♡) : je vous conseille de savoir faire cette question

rien : c'est le corps du travail

(*) : plus avancé, ou redondant avec des choses déjà vues

(**) : pour les élèves à la fois les plus intéressés et qui ont du temps

Attention ! L'énoncé d'une question « étoilée » peut être utile pour les questions suivantes et pourtant non-étoilées (c'est la *résolution* des questions étoilées qui porte une étoile, pas la lecture de leur énoncé !)

Règle du jeu sur le langage Python. L'annexe Python du programme officiel n'est pas exhaustive : vous avez le droit d'utiliser les fonctions intégrées (`sum`, `max`, `min`, `abs`, `sorted`, `int`...), les méthodes de listes comme `.count()` ou `.pop(i)`, le test `cle in dico`, etc.

Mais attention : on ne peut les utiliser que si elles ne sont **pas le cœur de la question**. Si l'on vous demande de « compter le nombre d'occurrences d'un objet dans une liste », répondre `liste.count(objet)` sera considéré comme de la triche. En revanche, si ce comptage n'est qu'un ingrédient d'une question plus vaste, `.count()` est le bienvenu.

Le module random

On utilise essentiellement les fonctions suivantes du module `random` (on les retrouve, avec quelques variantes, dans `numpy.random`) :

- `random()` : renvoie un réel pseudo-aléatoire de l'intervalle $[0,1[$, de telle sorte que les nombres obtenus suivent une loi uniforme sur cet intervalle (voir l'exercice 2).
- `randint(a, b)`, où `a` et `b` sont deux entiers : renvoie un entier de $\llbracket a; b \rrbracket$, avec une distribution uniforme. **Attention**, contrairement à `range(a, b)`, la borne `b` peut être atteinte.
- `choice(ma_liste)` : renvoie un élément de `ma_liste` choisi au hasard, de façon équirépartie. Pour un choix non équiréparti, on peut utiliser `numpy.random.choice(ma_liste, p = ...)`, où le paramètre `p` est la liste des probabilités voulues pour les différents éléments de `ma_liste`.
- `shuffle(ma_liste)` : mélange aléatoirement les éléments de `ma_liste`. Attention, cette fonction ne renvoie rien : elle travaille *en place*.

Les modules `random` et `numpy.random` disposent aussi de nombreuses lois non uniformes : `gauss(mu, sigma)`, `expovariate(lambd)`, etc. Je vous renvoie au poly sur les modules et types de données optionnels.

Deux façons d'importer. Dans un énoncé, vous pouvez rencontrer les deux syntaxes suivantes :

```

1 import random as rd          # on écrit ensuite rd.random(), rd.randint(1, 6)...
2 from random import random, randint  # on écrit ensuite random(), randint(1, 6)...

```

La première est la plus propre : on voit toujours de quel module vient chaque fonction, et on évite les collisions de noms (avec la seconde, le nom `random` désigne la *fonction*, plus le module). Il faut néanmoins savoir lire et utiliser les deux.

Dans ce TP, on utilise la première dans la partie I et la seconde dans la partie II.

Justification théorique

Tout ce TP repose sur l'idée suivante : si l'on répète N fois, de façon indépendante, une expérience aléatoire, la *fréquence* F_N de réalisation d'un événement A est proche de sa *probabilité* $p = P(A)$ quand N est grand.

Cela se démontrera en maths, dans les chapitres sur les variables aléatoires discrètes et sur les théorèmes limites. En voici le principe. Notons X_i la variable valant 1 si A est réalisé lors de la i -ème expérience, et 0 sinon. Les X_i sont des variables de Bernoulli de paramètre p , indépendantes.

Notons $F_N = \frac{1}{N} \sum_{i=1}^N X_i$: c'est la fréquence de réalisation de l'événement A sur N réalisations

La *loi faible des grands nombres* donne alors pour tout $\varepsilon > 0$:

$$P(|F_N - p| \geq \varepsilon) \xrightarrow{N \rightarrow +\infty} 0.$$

Autrement dit, quand le nombre d'expériences N est « grand », la fréquence d'apparition de A est *très probablement* une bonne approximation de $p = P(A)$.

NB : On peut montrer que l'écart entre fréquence et probabilité est de l'ordre de $\frac{1}{\sqrt{N}}$. Avec $N = 10\,000$ simulations, n'espérez pas mieux que deux décimales correctes !

I Probabilités

```
import random as rd
```

Correction de l'Exercice 1 Deux outils : moyenne et fréquence

```

1. 1 def moyenne(liste):
2     '''
3     Renvoie la moyenne des termes de liste (supposée non vide).
4     '''
5     return sum(liste) / len(liste)

```

Ou, si la question porte sur le calcul de la somme lui-même :

```

1 def moyenne(liste):
2     somme = 0          # somme des termes déjà parcourus
3     for terme in liste:
4         somme += terme
5     return somme / len(liste)

```

2. a) 1 `def frequence(liste, objet):`
 2 `'''`
 3 `Renvoie la fréquence de objet dans liste.`
 4 `liste.count(objet) est le nombre de fois où objet apparaît dans liste.`
 5 `'''`
 6 `return liste.count(objet) / len(liste)`

b) 1 `def nb_occurrences(liste, objet):`
 2 `'''`
 3 `Compte "à la main" le nombre de fois où objet apparaît dans liste.`
 4 `'''`
 5 `compteur = 0                      # nombre d'occurrences de objet rencontrées jusqu'ici`
 6 `for element in liste:`
 7 `if element == objet:`
 8 `compteur += 1`
 9 `return compteur`
 10
 11 `def frequence_a_la_main(liste, objet):`
 12 `return nb_occurrences(liste, objet) / len(liste)`

Les deux versions sont à connaître. Dans la suite du TP, le comptage n'est qu'un ingrédient : on utilisera donc sans scrupule `.count()`. En revanche, une question d'écrit du type « écrire une fonction qui compte le nombre d'occurrences de `objet` dans `liste` » attend la version à la main : répondre `liste.count(objet)` y serait considéré comme de la triche.

Correction de l'Exercice 2 *Les fonctions `random()` et `randint()` sont-elles uniformes ?*

1. 1 `def verif_random(x, y, nb_essais=100000):`
 2 `'''`
 3 `Estime la probabilité de l'événement "x <= rd.random() <= y"`
 4 `par sa fréquence sur nb_essais appels de rd.random().`
 5 `On doit trouver une valeur proche de y - x.`
 6 `'''`
 7 `x, y = min(x, y), max(x, y)       # on remet x et y dans le bon ordre`
 8 `nb_succes = 0                      # nombre d'appels tombés dans [x, y]`
 9 `for k in range(nb_essais):`
 10 `if x <= rd.random() <= y:`
 11 `nb_succes += 1`
 12 `return nb_succes / nb_essais`

Script de vérification sur quelques intervalles (un exemple de `print` par concaténation, à savoir écrire aussi) :

```
1 for (x, y) in [(0, 0.5), (0.2, 0.3), (0.1, 0.9), (0.75, 1)]:
2     print "[" + str(x) + ", " + str(y) + "]" : fréquence " + str(verif_random(x, y))
3         + " ; théorie " + str(y - x)
```

Exemple d'exécution :

```
[0, 0.5] : fréquence 0.50068 ; théorie 0.5
[0.2, 0.3] : fréquence 0.09902 ; théorie 0.099999999999999998
```

```
[0.1, 0.9] : fréquence 0.79981 ; théorie 0.8
[0.75, 1] : fréquence 0.2476 ; théorie 0.25
```

```
2. 1 def verif_randint(a, b, nb_essais=100000):
    2     '''
    3     Renvoie la liste des fréquences d'apparition de a, a+1, ..., b
    4     sur nb_essais appels de rd.randint(a, b).
    5     Chacune doit être proche de 1/(b-a+1).
    6     '''
    7     a, b = min(a, b), max(a, b)
    8     nb_occ = [0] * (b - a + 1)      # nb_occ[i] : nombre d'apparitions de la valeur a+i
    9     for k in range(nb_essais):
    10         r = rd.randint(a, b)        # résultat du tirage
    11         nb_occ[r - a] += 1          # la valeur r est comptée à l'indice r-a
    12     return [n / nb_essais for n in nb_occ]
```

```
>>> verif_randint(3, 7)
[0.1999, 0.20073, 0.20073, 0.19931, 0.19933]
```

Dans `verif_random`, remplacer `<=` par `<` ne change rien : la probabilité que `rd.random()` tombe exactement sur x ou sur y est nulle. Ce n'est évidemment pas le cas pour `randint()` !

Correction de l'Exercice 3 *Fabriquer `randint()` et `choice()`, lancer des dés*

- Idée :** posons $n = b - a + 1$, le nombre de valeurs que peut renvoyer `randint(a, b)`.
 - `rd.random()` est uniformément réparti sur $[0, 1[$, donc $n * \text{rd.random}()$ est uniformément réparti sur $[0, n[$;
 - sa partie entière `int(n * rd.random())` est donc un entier de $\llbracket 0; n-1 \rrbracket$, et chacune de ces valeurs a la même probabilité $\frac{1}{n}$ (c'est la longueur de l'intervalle $\left[\frac{k}{n}, \frac{k+1}{n}\right[$ dans lequel doit tomber `rd.random()`);
 - on veut une valeur entre a et b : il suffit d'ajouter a .

```
1 def mon_randint(a, b):
2     n = b - a + 1                # nombre de valeurs possibles
3     return a + int(n * rd.random())
```

On peut le tester avec la fonction `verif_randint` de l'exercice précédent, où l'on remplace `rd.randint` par `mon_randint`.

Attention : ici, `int()` est appliqué à un nombre positif, et renvoie bien sa partie entière. Mais `int()` se contente de « couper les décimales » :

```
>>> int(-3.5)
-3
```

alors que la partie entière de $-3,5$ est -4 . La vraie fonction partie entière est `floor` du module `math` :

```
>>> from math import floor
```

```
>>> floor(-3.5)
-4
```

C'est pour cela qu'on a écrit `a + int(n * rd.random())` et non `int(a + n * rd.random())`, qui serait faux pour $a < 0$.

2. **Idée** : on choisit au hasard un indice parmi les indices de la liste, puis on renvoie l'élément correspondant.

```
1 def mon_choice(maliste):
2     i = rd.randint(0, len(maliste) - 1)    # indice choisi au hasard
3     return maliste[i]
```

Attention! `rd.randint(a, b)` peut renvoyer b , d'où le « -1 ».

3.

```
1 def simule_de(n=6):
2     '''
3     Renvoie un entier entre 1 et n, tous les résultats étant équiprobables.
4     '''
5     return rd.randint(1, n)
```

4.

```
1 def somme_2_des():
2     return rd.randint(1, 6) + rd.randint(1, 6)
3     # ATTENTION ! Pas 2 * rd.randint(1, 6) : comprenez-vous pourquoi ?
```

- Théorie :

On modélise l'expérience par $\Omega = \llbracket 1; 6 \rrbracket^2$, muni de la probabilité uniforme (dés équilibrés et indépendants). Pour $s \in \llbracket 2; 12 \rrbracket$, le nombre de couples (i, j) tels que $i + j = s$ vaut $6 - |s - 7|$ (par exemple 1 couple pour $s = 2$, 6 couples pour $s = 7$). La probabilité d'obtenir la somme s vaut donc

$$\frac{6 - |s - 7|}{36}$$

- Vérification :

```
1 def verif_somme_2_des(nb_essais=100000):
2     '''
3     Affiche, pour chaque somme s possible, sa fréquence observée
4     et sa probabilité théorique.
5     '''
6     nb_occ = [0] * 13          # nb_occ[s] : nombre de fois où la somme vaut s
7                                # (les cases 0 et 1 restent à 0 : c'est juste plus possible)
8     for k in range(nb_essais):
9         nb_occ[somme_2_des()] += 1
10    for s in range(2, 13):
11        print("somme", s, ": fréquence", nb_occ[s] / nb_essais,
12              "; probabilité", (6 - abs(s - 7)) / 36)
```

Exemple d'exécution (début) :

```
somme 2 : fréquence 0.02702 ; probabilité 0.027777777777777776
somme 3 : fréquence 0.05629 ; probabilité 0.055555555555555555
somme 4 : fréquence 0.08356 ; probabilité 0.08333333333333333
```

Bien comprendre que, pour l'étude d'une expérience aléatoire, il y a d'une part la fonction qui *simule* l'expérience, et d'autre part la fonction (ou le script) qui *vérifie* que cette simulation est conforme à la théorie, en la lançant un grand nombre de fois.

Correction de l'Exercice 4 *Approximation de π par une méthode de Monte-Carlo*

Idée : la probabilité théorique de tomber dans le quart de disque est égale à son aire $\frac{\pi}{4}$, divisée par l'aire du carré dans lequel on tire (c'est-à-dire 1).

- On tire N fois au hasard x et y , indépendamment, à l'aide de `rd.random()`.
- On compte le nombre de fois où le point (x, y) est dans le quart de disque, c'est-à-dire où $x^2 + y^2 \leq 1$.
- La fréquence obtenue doit être proche de la probabilité théorique $\frac{\pi}{4}$: on la multiplie par 4 pour approcher π .

```
1 def pi_monte_carlo(N=100000):
2     dedans = 0          # nombre de points tombés dans le quart de disque
3     for k in range(N):
4         x = rd.random()    # abscisse du point tiré
5         y = rd.random()    # ordonnée du point tiré
6         if x**2 + y**2 <= 1:
7             dedans += 1
8     return 4 * dedans / N
```

```
>>> pi_monte_carlo()
3.14148
```

D'après la justification théorique donnée en début de TP, l'erreur est de l'ordre de $\frac{4}{\sqrt{N}}$: avec $N = 10^5$, on n'obtient guère que deux décimales de π . C'est une méthode très lente pour calculer π ... mais très robuste pour calculer des aires ou des volumes de formes compliquées, surtout en grande dimension.

Version 2, avec numpy (vectorisation). Le principe de la vectorisation sera expliqué dans un TP ultérieur ; voici seulement un avant-goût. Au lieu de tirer les points un par un dans une boucle, on tire d'un coup tout un tableau de N abscisses et un tableau de N ordonnées, et on calcule « terme à terme » sur ces tableaux. C'est beaucoup plus rapide.

```
1 import numpy as np
2
3 def pi_monte_carlo_np(N=1000000):
4     x = np.random.random(N)    # tableau de N abscisses uniformes dans [0, 1[
5     y = np.random.random(N)    # tableau de N ordonnées
6     dedans = np.sum(x**2 + y**2 <= 1)    # x**2 + y**2 <= 1 est un tableau de booléens
```

```

7                                     # np.sum compte les True
8     return 4 * dedans / N

```

Correction de l'Exercice 5 *Pile ou face*

- Idée :** à chaque lancer, on doit obtenir Pile avec probabilité p . D'après la propriété fondamentale de `rd.random()`, l'événement « `rd.random() < p` » (c'est-à-dire « `rd.random()` tombe dans $[0, p[$ ») a pour probabilité p : il convient donc parfaitement pour représenter « obtenir Pile ».

```

1  def lancers(n, p):
2      '''
3      Simule n lancers indépendants d'une pièce donnant Pile avec probabilité p.
4      Renvoie la liste des résultats : 1 pour Pile, 0 pour Face.
5      '''
6      resultats = []          # liste des résultats des lancers déjà effectués
7      for k in range(n):
8          if rd.random() < p:  # événement de probabilité p : "Pile"
9              resultats.append(1)
10         else:
11             resultats.append(0)
12     return resultats

```

Ou plus court, avec une liste en compréhension :

```

1  def lancers(n, p):
2      return [int(rd.random() < p) for k in range(n)]
3      # int(True) vaut 1 et int(False) vaut 0

```

Avec $p = \frac{1}{2}$, on peut aussi utiliser `rd.randint(0, 1)` pour simuler chaque lancer. Mais la méthode `rd.random() < p` est à connaître absolument : elle fonctionne pour tout p , et on va la réutiliser sans cesse.

- Les Pile étant représentés par des 1 et les Face par des 0, le nombre de Pile est la somme de la liste :

```

1  def nb_piles(n, p):
2      return sum(lancers(n, p))

```

Ou sans construire la liste :

```

1  def nb_piles(n, p):
2      compteur = 0          # nombre de Pile obtenus jusqu'ici
3      for k in range(n):
4          if rd.random() < p:
5              compteur += 1
6      return compteur

```

- a)

```

1  def estime_au_moins_un_pile(n, p, nb_essais=10000):
2      nb_succes = 0          # nombre de séries de n lancers contenant au moins n Pile
3      for k in range(nb_essais):

```

```

4         if 1 in lancers(n, p):
5             nb_succes += 1
6         return nb_succes / nb_essais

```

- b) Pour $i \in \llbracket 1; n \rrbracket$, notons P_i l'événement « le i -ème lancer donne Pile ». L'événement contraire de « obtenir au moins un Pile » est $\overline{P_1} \cap \dots \cap \overline{P_n}$. Les lancers étant **indépendants**, on a :

$$P(\overline{P_1} \cap \dots \cap \overline{P_n}) = P(\overline{P_1}) \times \dots \times P(\overline{P_n}) = (1 - p)^n,$$

donc

$$P(\text{obtenir au moins un Pile}) = 1 - (1 - p)^n$$

```

1 for (n, p) in [(3, 0.5), (5, 0.1), (10, 0.2)]:
2     print("n =", n, "p =", p, ": fréquence", estime_au_moins_un_pile(n, p),
3           "; probabilité", 1 - (1 - p)**n)

```

```

n = 3 p = 0.5 : fréquence 0.87 ; probabilité 0.875
n = 5 p = 0.1 : fréquence 0.4074 ; probabilité 0.40950999999999993
n = 10 p = 0.2 : fréquence 0.8924 ; probabilité 0.8926258175999999

```

Correction de l'Exercice 6 *Appelez la SPA !*

- On ajoute dès maintenant le paramètre `p` (qui vaut 0,5 par défaut), et un paramètre `affichage` pour pouvoir désactiver les messages dans la question suivante.

```

1 def appelez_la_spa(p=0.5, affichage=True):
2     '''
3     p : probabilité qu'Ariane touche la cible à chaque lancer (0.5 dans l'énoncé)
4     affichage : True si l'on veut le récit de la partie
5     Renvoie le nombre total de lancers.
6     '''
7     ariane = 0      # nombre de succès d'Ariane
8     boualem = 0     # nombre d'échecs d'Ariane (= points de Boualem)
9     while abs(ariane - boualem) <= 1:
10         if rd.random() < p:
11             ariane += 1
12             if affichage:
13                 print("MiaaaAAAOUUWW ! Pffffrrrrttt")
14         else:
15             boualem += 1
16             if affichage:
17                 print("Chtonk, le mur")
18     if affichage:
19         if ariane > boualem:
20             print("Ariane gagne")
21         else:
22             print("Boualem gagne")
23     return ariane + boualem

```

- On écrit une variante qui renvoie `True` si Ariane gagne, puis on estime la fréquence de ses victoires.

```

1 def spa_ariane_gagne(p=0.5):
2     '''
3     Simule une partie (sans affichage) et renvoie True si Ariane gagne.
4     '''
5     ariane = 0
6     boualem = 0
7     while abs(ariane - boualem) <= 1:
8         if rd.random() < p:
9             ariane += 1
10        else:
11            boualem += 1
12    return ariane > boualem
13
14 def estime_proba_ariane(p=0.5, nb_essais=10000):
15     nb_victoires = 0      # nombre de parties gagnées par Ariane
16     for k in range(nb_essais):
17         if spa_ariane_gagne(p):
18             nb_victoires += 1
19     return nb_victoires / nb_essais

```

```

1 for p in [0.5, 0.6, 0.8]:
2     print("p =", p, ": fréquence", estime_proba_ariane(p),
3           "; théorie", p**2 / (p**2 + (1 - p)**2))

```

```

p = 0.5 : fréquence 0.5039 ; théorie 0.5
p = 0.6 : fréquence 0.6917 ; théorie 0.6923076923076923
p = 0.8 : fréquence 0.9446 ; théorie 0.9411764705882353

```

3. Notons $q = 1 - p$, et pour tout $i \geq 1$, S_i l'événement « Ariane touche la cible au i -ème lancer ». Les S_i sont indépendants et de probabilité p .

- Déroulement d'une partie par paires de lancers :

Regroupons les lancers par paires : la k -ème paire est formée des lancers $2k - 1$ et $2k$. Notons M_k l'événement « la k -ème paire est mixte » (un succès et un échec), c'est-à-dire

$$M_k = (S_{2k-1} \cap \overline{S_{2k}}) \cup (\overline{S_{2k-1}} \cap S_{2k}).$$

Si la partie n'est pas finie avant la k -ème paire, l'écart entre succès et échecs vaut 0 au début de cette paire. Après le lancer $2k - 1$, il vaut ± 1 : la partie continue. Après le lancer $2k$:

- soit la paire a donné deux succès : l'écart vaut $+2$, Ariane gagne ;
- soit la paire a donné deux échecs : l'écart vaut -2 , Boualem gagne ;
- soit la paire est mixte : l'écart revient à 0, la partie continue.

Ainsi, l'événement A_k = « Ariane gagne au lancer $2k$ » s'écrit

$$A_k = M_1 \cap \dots \cap M_{k-1} \cap S_{2k-1} \cap S_{2k}.$$

- Probabilité de A_k :

La réunion définissant M_k est disjointe et les lancers sont indépendants, donc $P(M_k) = pq + qp = 2pq$. Les événements $M_1, \dots, M_{k-1}, S_{2k-1}, S_{2k}$ portent sur des lancers distincts : par **indépendance** des lancers (lemme des coalitions),

$$P(A_k) = (2pq)^{k-1} p^2$$

• Conclusion :

L'événement « Ariane gagne » est la réunion des A_k pour $k \geq 1$, et cette réunion est **disjointe** (deux événements A_k et A_ℓ avec $k \neq \ell$ sont incompatibles : la partie s'arrête à un seul instant). Comme $0 \leq 2pq \leq \frac{1}{2} < 1$, la série géométrique converge et :

$$P(\text{Ariane gagne}) = \sum_{k=1}^{+\infty} p^2 (2pq)^{k-1} = \frac{p^2}{1 - 2pq}.$$

Enfin, $1 - 2pq = (p + q)^2 - 2pq = p^2 + q^2$, donc

$$P(\text{Ariane gagne}) = \frac{p^2}{p^2 + (1 - p)^2}$$

De même, la probabilité que Boualem gagne vaut $\frac{q^2}{p^2 + q^2}$. La somme des deux vaut 1 : l'événement « la partie ne s'arrête jamais » est donc quasi-impossible. Pour $p = \frac{1}{2}$, on retrouve bien une chance sur deux pour chacun (heureusement, par symétrie!).

4. • Simulation :

On écrit une variante de `appelez_la_spa` qui renvoie seulement la durée, sans affichage (on aurait aussi pu appeler `appelez_la_spa(p, False)`).

```

1  def spa_duree(p=0.5):
2      '''
3      Simule une partie (sans affichage) et renvoie le nombre total de lancers.
4      '''
5      ariane = 0
6      boualem = 0
7      while abs(ariane - boualem) <= 1:
8          if rd.random() < p:
9              ariane += 1
10         else:
11             boualem += 1
12     return ariane + boualem
13
14 def estime_proba_duree(k, p=0.5, nb_essais=10000):
15     nb_longues = 0      # nombre de parties ayant duré plus de 2k lancers
16     for i in range(nb_essais):
17         if spa_duree(p) > 2 * k:
18             nb_longues += 1
19     return nb_longues / nb_essais
20
21 for (k, p) in [(1, 0.5), (3, 0.5), (2, 0.7)]:
22     print("k =", k, "p =", p, ": fréquence", estime_proba_duree(k, p),
23           "; théorie", (2 * p * (1 - p))**k)

```

```

k = 1 p = 0.5 : fréquence 0.4901 ; théorie 0.5
k = 3 p = 0.5 : fréquence 0.128 ; théorie 0.125
k = 2 p = 0.7 : fréquence 0.1796 ; théorie 0.17640000000000003

```

• Démonstration :

D'après l'étude de la question 3, la partie dure strictement plus de $2k$ lancers si et seulement

si les k premières paires de lancers sont mixtes. Avec les notations de cette question, et par **indépendance** des lancers (lemme des coalitions) :

$$P(\text{la partie dure plus de } 2k \text{ lancers}) = P(M_1 \cap \dots \cap M_k) = (2pq)^k$$

5. a) Comme p et $q = 1 - p$ sont strictement positifs :

$$\begin{aligned} \frac{p^2}{p^2 + q^2} = \frac{9}{10} &\iff 10p^2 = 9p^2 + 9q^2 \iff p^2 = 9q^2 \\ &\iff p = 3q \iff p = 3(1 - p) \iff p = \frac{3}{4}. \end{aligned}$$

$$p = \frac{3}{4}$$

- b) Notons $f : p \mapsto \frac{p^2}{p^2 + (1 - p)^2}$. Pour $p \in \left[\frac{1}{2}, 1\right]$, $\frac{1}{f(p)} = 1 + \left(\frac{1}{p} - 1\right)^2$, et $\frac{1}{p} - 1$ est positive et strictement décroissante sur cet intervalle : $\frac{1}{f}$ y est strictement décroissante, donc f strictement croissante.

La fonction f est strictement croissante et continue sur l'intervalle $\left[\frac{1}{2}, 1\right]$, avec $f\left(\frac{1}{2}\right) = \frac{1}{2}$ et $f(1) = 1$, donc f établit une bijection de $\left[\frac{1}{2}, 1\right]$ sur $\left[\frac{1}{2}, 1\right]$. Comme f est bijective et que `cible` appartient à $\left[\frac{1}{2}, 1\right]$, l'équation $f(p) = \text{cible}$ admet une et une seule solution sur $\left[\frac{1}{2}, 1\right]$.

On l'approche par dichotomie : on garde à chaque étape la moitié de l'intervalle qui contient la solution.

```

1  def proba_ariane(p):
2      return p**2 / (p**2 + (1 - p)**2)
3
4  def dichotomie(cible, a=0.5, b=1, epsilon=1e-6):
5      '''
6      Renvoie une valeur approchée à epsilon près de l'unique p de [a, b]
7      tel que proba_ariane(p) = cible (proba_ariane est croissante sur [a, b]).
8      '''
9      while b - a > epsilon:
10         c = (a + b) / 2           # milieu de l'intervalle
11         if proba_ariane(c) < cible: # la solution est dans [c, b]
12             a = c
13         else:                     # la solution est dans [a, c]
14             b = c
15     return (a + b) / 2

```

```

>>> dichotomie(0.9)
0.7499995231628418
>>> dichotomie(0.99)
0.9086747169494629

```

Pour gagner avec probabilité 0,99, Ariane doit toucher la cible dans environ 91 % des cas (la valeur exacte est $\frac{\sqrt{99}}{1 + \sqrt{99}}$).

6. a) 1 `def spa_ecart(p=0.5, m=2):`
 2 `'''`
 3 `Simule une partie à écart m (sans affichage) ; renvoie True si Ariane gagne.`
 4 `'''`
 5 `ariane = 0`
 6 `boualem = 0`
 7 `while abs(ariane - boualem) < m:`
 8 `if rd.random() < p:`
 9 `ariane += 1`
 10 `else:`
 11 `boualem += 1`
 12 `return ariane > boualem`
 13
 14 `def estime_proba_ariane_ecart(p=0.5, m=2, nb_essais=10000):`
 15 `nb_victoires = 0`
 16 `for i in range(nb_essais):`
 17 `if spa_ecart(p, m):`
 18 `nb_victoires += 1`
 19 `return nb_victoires / nb_essais`

b) 1 `for m in [1, 2, 3, 4]:`
 2 `print("m =", m, ":", estime_proba_ariane_ecart(0.6, m))`

```
m = 1 : 0.6103
m = 2 : 0.6939
m = 3 : 0.7652
m = 4 : 0.8335
```

On conjecture :

$$P(\text{Ariane gagne}) = \frac{p^m}{p^m + (1-p)^m}$$

qui donne, pour $p = 0,6$, les valeurs 0,6 ; 0,692 ; 0,771 ; 0,835. Pour $m = 1$, c'est évident, et pour $m = 2$, c'est la question 3.

La conjecture est vraie : c'est un cas particulier du problème de la *ruine du joueur* (l'écart entre succès et échecs effectuée une *marche aléatoire*, qui part de 0 et s'arrête en $-m$ ou en m). Sa démonstration dépasse le cadre de ce TP.

Pourquoi deux points d'écart au tennis ? À partir de « égalité » (40-40), un jeu de tennis se déroule exactement comme la SPA : il faut deux points d'écart pour gagner. Pour un joueur qui gagne 60% des points, exiger deux points d'écart fait passer sa probabilité de l'emporter de 0,6 à 0,69, et un écart de 3 la porterait à 0,77. Exiger un écart amplifie la différence de niveau : le meilleur joueur gagne plus souvent, et le hasard pèse moins.

Correction de l'Exercice 7 (**) *Le tirage au sort des cadeaux*

1. **Idée** : on représente le chapeau par la liste des noms qui y restent. Chaque personne tire au hasard un indice dans cette liste, et le nom correspondant est retiré du chapeau grâce à la méthode `.pop(j)`, qui enlève l'élément d'indice j de la liste **et le renvoie**.

```
1 def tirage(n):  
2 '''
```

```

3     Simule le tirage au sort : renvoie la liste t telle que
4     t[i] est le nom tiré par la personne i.
5     '''
6     chapeau = [i for i in range(n)]      # noms encore dans le chapeau
7     resultat = []                       # noms tirés successivement par 0, 1, .
8     for personne in range(n):
9         j = rd.randint(0, len(chapeau) - 1)    # indice du papier tiré dans le chapeau
10        resultat.append(chapeau.pop(j))        # on retire ce papier du chapeau
11    return resultat

```

On pourrait aussi mélanger la liste $[0, 1, \dots, n-1]$ avec `rd.shuffle` (voir le cadre sur le module `random`), mais l'idée de ce TP est de simuler au plus près l'expérience décrite.

2. On parcourt le tirage, et on s'arrête dès qu'une personne a tiré son propre nom :

```

1    def est_reussi(t):
2        for i in range(len(t)):
3            if t[i] == i:      # la personne i a tiré son propre nom
4                return False
5        return True          # on n'a trouvé personne ayant tiré son propre nom

```

3.

```

1    def estime_proba_reussite(n, nb_essais=10000):
2        nb_reussites = 0      # nombre de tirages réussis
3        for k in range(nb_essais):
4            if est_reussi(tirage(n)):
5                nb_reussites += 1
6        return nb_reussites / nb_essais
7
8    from math import exp
9
10   for n in range(2, 13):
11       print("n =", n, ": fréquence de réussite", estime_proba_reussite(n))
12   print("exp(-1) =", exp(-1))

```

```

n = 2 : fréquence de réussite 0.491
n = 3 : fréquence de réussite 0.3307
n = 4 : fréquence de réussite 0.3768
n = 5 : fréquence de réussite 0.3717
n = 6 : fréquence de réussite 0.363
...
n = 12 : fréquence de réussite 0.3707
exp(-1) = 0.36787944117144233

```

Dès que $n \geq 4$ ou 5, la probabilité de réussite semble se stabiliser autour de 0,37, ce qui laisse penser que p_n tend vers e^{-1} . Autrement dit, quel que soit le nombre de personnes, il y a presque deux chances sur trois de devoir recommencer le tirage !

4. a) Le résultat du tirage est la liste des noms tirés successivement, sans remise, par les personnes $1, 2, \dots, n$: c'est une liste sans répétition des n noms. Le tirage étant fait « au hasard », ces $n!$ listes sont équiprobables : on munit l'ensemble de ces listes de la probabilité uniforme. Les tirages réussis en sont d_n , donc

$$p_n = \frac{d_n}{n!}$$

Avec une seule personne, elle tire forcément son propre nom : $\boxed{d_1 = 0}$. Avec deux personnes, la seule attribution réussie est l'échange des deux noms : $\boxed{d_2 = 1}$. Avec trois personnes, la personne 1 tire le nom 2 ou le nom 3 ; si elle tire 2, la personne 3 ne peut tirer que 1 (sinon elle aurait son propre nom), et la personne 2 reçoit 3 ; de même si elle tire 3. D'où $\boxed{d_3 = 2}$.

b) Soit $n \geq 3$. Une attribution réussie est déterminée de la façon suivante.

- On choisit le nom k tiré par la personne n : $n - 1$ possibilités (tous sauf n),
- PUIS on complète l'attribution :
 - SOIT la personne k a tiré le nom n (les personnes k et n « échangent leurs cadeaux »). Les $n - 2$ autres personnes se répartissent alors les $n - 2$ autres noms, sans qu'aucune ne tire son propre nom : d_{n-2} possibilités ;
 - SOIT la personne k n'a pas tiré le nom n . Notons j la personne qui a tiré le nom n (on a $j \neq n$ et $j \neq k$). Faisons sortir la personne n du jeu, en donnant à j le nom k à la place du nom n : on obtient une attribution des noms $1, \dots, n - 1$ aux personnes $1, \dots, n - 1$ dans laquelle personne n'a son propre nom (en particulier, j reçoit $k \neq j$). Réciproquement, partant d'une telle attribution, on retrouve l'attribution initiale : la personne j qui a reçu le nom k reçoit à la place le nom n , et la personne n reçoit le nom k . Cette correspondance est donc une bijection : d_{n-1} possibilités.

Ces deux cas sont disjoints : $d_{n-2} + d_{n-1}$ possibilités.

Conclusion : D'après le lemme des bergers, le nombre d'attributions réussies vaut :

$$\boxed{d_n = (n - 1) (d_{n-1} + d_{n-2})}$$

On vérifie que $d_3 = 2(d_2 + d_1) = 2$.

c) Soit $n \geq 3$. En divisant la relation précédente par $n!$:

$$p_n = \frac{(n - 1) d_{n-1}}{n!} + \frac{(n - 1) d_{n-2}}{n!} = \frac{n - 1}{n} \frac{d_{n-1}}{(n - 1)!} + \frac{1}{n} \frac{d_{n-2}}{(n - 2)!} = \left(1 - \frac{1}{n}\right) p_{n-1} + \frac{1}{n} p_{n-2},$$

donc

$$\boxed{p_n - p_{n-1} = -\frac{1}{n} (p_{n-1} - p_{n-2})}$$

Montrons alors par récurrence que pour tout $n \geq 2$, $p_n - p_{n-1} = \frac{(-1)^n}{n!}$.

— Pour $n = 2$: $p_2 - p_1 = \frac{1}{2} - 0 = \frac{(-1)^2}{2!}$.

— Si la propriété est vraie au rang $n - 1 \geq 2$, alors d'après la relation précédente

$$p_n - p_{n-1} = -\frac{1}{n} \times \frac{(-1)^{n-1}}{(n - 1)!} = \frac{(-1)^n}{n!}.$$

Conclusion : Pour tout $n \geq 2$, $\boxed{p_n - p_{n-1} = \frac{(-1)^n}{n!}}$.

d) Soit $n \geq 2$. Par télescopage, puisque $p_1 = 0$:

$$p_n = p_1 + \sum_{k=2}^n (p_k - p_{k-1}) = \sum_{k=2}^n \frac{(-1)^k}{k!} = \sum_{k=0}^n \frac{(-1)^k}{k!},$$

car les termes d'indices $k = 0$ et $k = 1$ valent 1 et -1 , et se compensent.

On reconnaît la somme partielle de la série exponentielle $\sum \frac{x^k}{k!}$ en $x = -1$, qui converge et a pour somme e^{-1} . Ainsi :

$$p_n \xrightarrow{n \rightarrow +\infty} e^{-1} \approx 0,368$$

- e) On calcule la somme en mettant à jour le terme $\frac{(-1)^k}{k!}$ d'une itération à l'autre, ce qui évite de calculer des factorielles :

```

1  def proba_exacte(n):
2      terme = 1          # contient (-1)^k / k! ; vaut 1 pour k = 0
3      p = 1              # somme des termes pour les indices de 0 à k
4      for k in range(1, n + 1):
5          terme = -terme / k      # passage de (-1)^(k-1)/(k-1)! à (-1)^k/k!
6          p += terme
7      return p
8
9  for n in range(2, 13):
10     print("n =", n, ": estimation", estime_proba_reussite(n),
11           "; valeur exacte", proba_exacte(n))

```

```

n = 2 : estimation 0.5009 ; valeur exacte 0.5
n = 3 : estimation 0.3294 ; valeur exacte 0.3333333333333333
n = 4 : estimation 0.3775 ; valeur exacte 0.3750000000000000
n = 5 : estimation 0.3706 ; valeur exacte 0.3666666666666667
...

```

La convergence est très rapide : on peut montrer que $|p_n - e^{-1}| \leq \frac{1}{(n+1)!}$, si bien que pour $n \geq 7$, p_n et e^{-1} coïncident déjà sur 4 décimales. On remarque d'ailleurs sur l'exemple d'exécution que les fluctuations de l'estimation (de l'ordre de $\frac{1}{\sqrt{10\,000}} = 0,01$) sont bien plus grandes que les variations de p_n . Un tirage avec 8 personnes ou avec 80 a donc pratiquement la même probabilité de réussite. Ce problème est un grand classique, connu sous le nom de *problème des rencontres* (ou *problème du chapelier*). Les attributions où personne ne reçoit son propre nom s'appellent des *dérangements*.

Correction de l'Exercice 8 (**) *Démasquer les tricheurs : les séries de Pile*

1. On parcourt la liste en tenant à jour la longueur de la série de 1 en cours, et le record.

```

1  def plus_longue_serie(resultats):
2      '''
3      Renvoie la longueur de la plus longue suite de 1 consécutifs dans resultats
4      '''
5      record = 0      # longueur de la plus longue série de 1 rencontrée jusqu'ic
6      serie = 0       # longueur de la série de 1 en cours
7      for x in resultats:
8          if x == 1:      # la série en cours se prolonge
9              serie += 1
10             if serie > record:
11                 record = serie
12             else:
13                 # un 0 interrompt la série en cours
14                 serie = 0
15     return record

```

2. On réutilise la fonction `lancers` de l'exercice 5.

```

1 def estime_proba_serie(n, k, p=0.5, nb_essais=10000):
2     nb_succes = 0          # nombre de simulations contenant une série de k Pile
3     for i in range(nb_essais):
4         if plus_longue_serie(lancers(n, p)) >= k:
5             nb_succes += 1
6     return nb_succes / nb_essais
7
8 for k in range(4, 11):
9     print("k =", k, ": estimation", estime_proba_serie(200, k))

```

```

k = 4 : estimation 0.9994
k = 5 : estimation 0.9667
k = 6 : estimation 0.7952
k = 7 : estimation 0.548
k = 8 : estimation 0.3223
k = 9 : estimation 0.1726
k = 10 : estimation 0.0915

```

Sur 200 lancers d'une pièce équilibrée, on obtient presque toujours une série de 5 Pile, et plus d'une fois sur deux une série de 7 Pile !

3. Si la pièce est équilibrée, la probabilité de ne jamais obtenir plus de 4 Pile d'affilée en 200 lancers est $1 - P(A_{200})$ pour $k = 5$, c'est-à-dire environ $1 - 0,966 = 0,034$ d'après la question précédente. C'est possible, mais très peu probable : on a de bonnes raisons de soupçonner une suite inventée.

On peut en faire une expérience en classe : avant le TP, la moitié des élèves lance réellement une pièce 200 fois et note les résultats, l'autre moitié invente une suite de 200 résultats. En regardant seulement la plus longue série de Pile (ou de Face), on démasque la plupart des tricheurs.

4. a) Pour $n < k$, on ne peut pas obtenir de série de k Pile en n lancers : $\boxed{u_n = 1}$ (y compris $u_0 = 1$). Pour $n = k$, la seule façon d'obtenir une série de k Pile est que les k lancers donnent Pile, ce qui a pour probabilité p^k par **indépendance** des lancers : $\boxed{u_k = 1 - p^k}$.

b) • Système complet :

Les B_j sont deux à deux incompatibles : si $j < j'$, B_j impose que le lancer j donne Face, et $B_{j'}$ qu'il donne Pile. Et l'un d'eux est toujours réalisé : soit l'un des k premiers lancers donne Face, et si j est le rang du premier d'entre eux, B_j est réalisé ; soit aucun ne donne Face, et B_{k+1} est réalisé. Ces événements ont un sens car $n \geq k$.

• Cas de B_{k+1} :

Si B_{k+1} est réalisé, les k premiers lancers forment une série de k Pile, donc A_n est réalisé : $B_{k+1} \cap \overline{A_n} = \emptyset$, et $\boxed{P(B_{k+1} \cap \overline{A_n}) = 0}$.

• Cas de B_j , pour $j \in \llbracket 1 ; k \rrbracket$:

Notons C_j l'événement « parmi les lancers numéros $j + 1$ à n , il n'y a pas de série de k Pile ». Supposons B_j réalisé. Le lancer j donne Face, donc une série de k Pile ne peut pas le contenir. Elle ne peut pas non plus être située avant lui, puisqu'il n'y a que $j - 1 < k$ lancers avant. Une éventuelle série de k Pile est donc située parmi les lancers $j + 1$ à n . Ainsi :

$$B_j \cap \overline{A_n} = B_j \cap C_j.$$

L'événement B_j ne dépend que des j premiers lancers, et C_j des lancers $j + 1$ à n . Par **indépendance** des lancers (lemme des coalitions) :

$$P(B_j \cap \overline{A_n}) = P(B_j) P(C_j).$$

Toujours par indépendance, $P(B_j) = p^{j-1}q$. Enfin, les lancers $j + 1$ à n forment une suite de $n - j$ lancers indépendants de la même pièce : la probabilité qu'ils ne contiennent pas de série de k Pile est la même que pour les $n - j$ premiers lancers, c'est-à-dire u_{n-j} . Ainsi :

$$P(B_j \cap \overline{A_n}) = p^{j-1}q u_{n-j}$$

- c) Soit $n \geq k$. D'après la formule des probabilités totales appliquée au système complet $(B_1, \dots, B_{k+1})$:

$$u_n = P(\overline{A_n}) = \sum_{j=1}^{k+1} P(B_j \cap \overline{A_n}),$$

donc d'après la question précédente :

$$\forall n \geq k, u_n = \sum_{j=1}^k p^{j-1}q u_{n-j}$$

On vérifie la cohérence avec la question **a** : $u_k = \sum_{j=1}^k p^{j-1}q = q \frac{1-p^k}{1-p} = 1 - p^k$.

- d) On stocke les u_m dans une liste, en partant de $u_0 = \dots = u_{k-1} = 1$.

```

1  def proba_serie(n, k, p=0.5):
2      '''
3      Renvoie la probabilité d'obtenir au moins une série de k Pile en n lancers.
4      '''
5      q = 1 - p
6      u = [1] * k          # u[m] contient u_m ; u_m = 1 pour m < k
7      for m in range(k, n + 1):
8          s = 0             # somme de la relation de récurrence
9          for j in range(1, k + 1):
10             s += p**(j - 1) * q * u[m - j]
11             u.append(s)    # u[m] contient maintenant u_m
12     return 1 - u[n]
13
14 for k in range(4, 11):
15     print("k =", k, ": estimation", estime_proba_serie(200, k),
16           "; valeur exacte", proba_serie(200, k))

```

```

k = 4 : estimation 0.9994 ; valeur exacte 0.9993180402100673
k = 5 : estimation 0.9667 ; valeur exacte 0.9659033812596826
k = 6 : estimation 0.7952 ; valeur exacte 0.8009321842394238
k = 7 : estimation 0.548 ; valeur exacte 0.5437079159158521
...

```

La relation de récurrence de la question **4c** est une généralisation de la suite de Fibonacci : pour $k = 2$ et $p = \frac{1}{2}$, elle s'écrit $u_n = \frac{1}{2}u_{n-1} + \frac{1}{4}u_{n-2}$, et $2^n u_n$ est alors le nombre de suites de n Pile ou Face sans deux Pile consécutifs... qui est un nombre de Fibonacci.

II Variables aléatoires discrètes

Pour varier les plaisirs, on commence le fichier de cette deuxième partie avec la ligne :

```
from random import randint, random, choice
```

Simuler une variable aléatoire : de quoi parle-t-on ? D'une manière générale, « simuler une variable aléatoire X » consiste à écrire une fonction `simule_X()` qui renvoie un élément de $X(\Omega)$, de telle sorte que pour tout $v \in X(\Omega)$, la probabilité que `simule_X()` renvoie v soit égale à $P(X = v)$. Pour « vérifier expérimentalement les résultats théoriques sur la loi de X », on lance la fonction `simule_X()` un grand nombre de fois, on calcule la fréquence de chaque valeur obtenue, et on vérifie que ces fréquences sont proches des probabilités théoriques données par la loi de X .

Attention ! Pour *simuler* X , on ne veut pas utiliser sa loi théorique, mais rester au plus près de la *définition* de X . Par exemple, pour simuler une variable binomiale, on ne veut pas voir de $\binom{n}{k} p^k (1-p)^{n-k}$ dans le code, mais réellement simuler n épreuves de Bernoulli et compter les succès. Fondamentalement, notre but est de fabriquer à l'aide du module `random` une *machine qui se comporte du point de vue probabiliste de la même manière que le phénomène qu'on cherche à simuler*, et NON calculer en python les probas données par la théorie.

Vocabulaire : Moyenne, Variance et Fonction de Répartition empiriques

Soit X une variable aléatoire discrète admettant une espérance et variance. Soient $X_1, \dots, X_N$ des variables aléatoires indépendantes de même loi que X (en pratique ici : les résultats de N simulations de X). On définit :

- la *moyenne empirique* $M_N = \frac{1}{N} \sum_{i=1}^N X_i$. La *loi faible des grands nombres* (vue plus tard dans l'année, et qu'on a déjà utilisée implicitement dans la **Partie I**) donne que pour N « grand », M_N donne une *estimation* de $E(X)$
- la *variance empirique* $\frac{1}{N} \sum_{i=1}^N (X_i - M_N)^2$ qui donne de même une estimation de $V(X)$.
- pour tout réel t , la fréquence de l'événement $(X_k \leq t)$ au cours des N simulations, appelée *fonction de répartition empirique* et notée $F_N(t)$, estime $F_X(t) = P(X \leq t)$ (c'est exactement la situation de la partie I, avec l'événement $(X \leq t)$).

« **Rappels** » de tracé (ou cf. le poly Annexe III du cours de Python, et le TP correspondant !). On utilise le module `matplotlib`, qu'on importe par `import matplotlib.pyplot as plt`. Rappels :

- `plt.plot(liste_x, liste_y)` crée la ligne brisée reliant les points dont les abscisses et les ordonnées sont données respectivement par les listes `liste_x` et `liste_y`. Paramètres optionnels utiles : `color='red'`, `linestyle='--'`, `marker='o'` (pour marquer les points), `label='...'` (pour la légende).
- `plt.bar(liste_x, liste_hauteurs, width=0.5)` crée un diagramme en bâtons : une barre de largeur 0,5 centrée en chaque abscisse, de hauteur donnée. C'est l'outil idéal pour représenter des fréquences.
- `plt.step(liste_x, liste_y, where='post')` crée une fonction en escalier, constante entre deux abscisses consécutives : idéal pour une fonction de répartition.
- `plt.legend()` crée la légende ;
- `plt.show()` affiche sur la même figure tous les éléments définis par les commandes précédentes

Correction de l'Exercice 9 Variables de Bernoulli, variables binomiales

1. **Idée** : d'après la propriété fondamentale de `random()`, l'événement « `random() < p` », c'est-à-dire « `random()` tombe dans $[0, p[$ », a pour probabilité p .

```
1 def bernoulli(p):
2     '''
3     Simule une variable de Bernoulli de paramètre p : renvoie 1 avec
4     probabilité p, et 0 avec probabilité 1 - p.
5     '''
6     if random() < p:
7         return 1
8     else:
9         return 0
```

On peut aussi décider de renvoyer `True` pour 1 et `False` pour 0, en remplaçant les quatre dernières lignes par un simple `return random() < p`.

• Vérification :

On lance la fonction un grand nombre de fois, et on vérifie que la fréquence des 1 est proche de p , pour plusieurs valeurs de p . On réutilise la fonction `frequence` de l'exercice 1.

```
1 N = 10000          # nombre de simulations
2 for p in [0.1, 0.3, 0.5, 0.9]:
3     echantillon = [bernoulli(p) for k in range(N)] # résultats des N simulations
4     print("p = " + str(p) + " ; fréquence de 1 : " + str(frequence(echantillon, N)))
```

```
p = 0.1 ; fréquence de 1 : 0.0946
p = 0.3 ; fréquence de 1 : 0.3034
p = 0.5 ; fréquence de 1 : 0.4989
p = 0.9 ; fréquence de 1 : 0.9044
```

2. Une variable de loi $\mathcal{B}(n, p)$ compte le nombre de succès dans une suite de n épreuves de Bernoulli indépendantes de même paramètre p :

```
1 def variable_binomiale(n, p):
2     '''
3     Simule une variable de loi B(n, p) : nombre de succès
4     au cours de n épreuves de Bernoulli indépendantes de paramètre p.
5     '''
6     nb_succes = 0          # nombre de succès obtenus jusqu'ici
7     for k in range(n):
8         nb_succes += bernoulli(p)
9     return nb_succes
```

C'est exactement la fonction `nb_piles` de l'exercice 5!

3. a) On utilise
$$\binom{n}{k} = \frac{n(n-1)\cdots(n-k+1)}{k!} = \frac{\prod_{i=1}^k (n-k+i)}{\prod_{i=1}^k i} :$$

```
1 def k_parmi_n(k, n):
2     '''
3     Renvoie le coefficient binomial "k parmi n" (0 si k < 0 ou k > n).
4     '''
```

```

5     if k < 0 or k > n:
6         return 0
7     numerateur = 1      # contiendra n(n-1)...(n-k+1)
8     denominateur = 1    # contiendra k!
9     for i in range(1, k + 1):
10        numerateur *= n - k + i
11        denominateur *= i
12    return numerateur // denominateur    # division exacte entre entiers

```

La division entière `//` est ici exacte, et elle a l'avantage de renvoyer un entier (alors que `/` renverrait un flottant). Notons que la fonction `comb(n, k)` du module `math` fait ce travail.

b)

```

1 def proba_binomiale(n, p, k):
2     '''
3     Renvoie P(X = k) pour X de loi B(n, p).
4     '''
5     return k_parmi_n(k, n) * p**k * (1 - p)**(n - k)

```

- c) On lance un grand nombre de fois la fonction `variable_binomiale`, en stockant les résultats dans une liste. Puis, pour chaque $k \in \llbracket 0; n \rrbracket$, on calcule d'une part la valeur théorique de $P(X = k)$, d'autre part la fréquence d'apparition de k dans la liste des résultats.

```

1 def compare_binomiale(n=10, p=0.3, nb_essais=10000):
2     essais = [variable_binomiale(n, p) for i in range(nb_essais)] # simulations
3     for k in range(n + 1):
4         print("k =", k, "; théorie :", proba_binomiale(n, p, k),
5               "; expérience :", frequence(essais, k))

```

```

>>> compare_binomiale()
k = 0 ; théorie : 0.028247524899999984 ; expérience : 0.0308
k = 1 ; théorie : 0.121060820999999993 ; expérience : 0.1237
k = 2 ; théorie : 0.233474440499999988 ; expérience : 0.2341
k = 3 ; théorie : 0.26682793199999998 ; expérience : 0.2595
...

```

Et pour tester plusieurs valeurs de n et de p , on fait des boucles imbriquées :

```

1 for n in [2, 5, 10, 20]:
2     for p in [0.1, 0.3, 0.6, 0.9]:
3         print("----- n =", n, "et p =", p)
4         compare_binomiale(n, p)

```

4. a)

```

1 def variance_empirique(liste):
2     m = moyenne(liste)      # moyenne empirique
3     somme = 0                # somme des carrés des écarts à la moyenne
4     for x in liste:
5         somme += (x - m)**2
6     return somme / len(liste)

```

- b) On rappelle que si $X \hookrightarrow \mathcal{B}(n, p)$, alors $E(X) = np$ et $V(X) = np(1 - p)$.

```

1 n, p = 10, 0.3
2 N = 10000

```

```

3  echantillon = [variable_binomiale(n, p) for k in range(N)]
4  print("moyenne empirique :", moyenne(echantillon), "; espérance :", n * p)
5  print("variance empirique :", variance_empirique(echantillon),
6        "; variance :", n * p * (1 - p))

```

```

moyenne empirique : 3.0222 ; espérance : 3.0
variance empirique : 2.1971071600000274 ; variance : 2.0999999999999996

```

Version 2, avec numpy (le principe de la vectorisation sera expliqué dans un TP ultérieur).

```

1  import numpy as np
2
3  def binomiale_np(n, p):
4      # np.random.random(n) < p est un tableau de n booléens, chacun valant True
5      # avec probabilité p : np.sum compte les True, c'est-à-dire les succès
6      return np.sum(np.random.random(n) < p)
7
8  def echantillon_binomiale_np(n, p, N=10000):
9      # tableau de N lignes et n colonnes : chaque ligne représente n épreuves
10     tirages = np.random.random((N, n)) < p
11     return np.sum(tirages, axis=1)      # somme de chaque ligne : N simulations de (n, p)
12
13     e = echantillon_binomiale_np(10, 0.3)
14     print(np.mean(e), np.var(e))      # moyenne et variance empiriques

```

Enfin, `np.random.binomial(n, p)` simule directement une variable de loi $\mathcal{B}(n, p)$ (et `np.random.binomial(1, p)` une variable de Bernoulli).

Correction de l'Exercice 10 *Simuler une loi géométrique*

1. Une variable de loi $\mathcal{G}(p)$ est le rang du premier succès dans une suite d'épreuves de Bernoulli indépendantes de paramètre p : on effectue des épreuves tant qu'on n'a pas obtenu de succès, en comptant les épreuves.

```

1  def variable_geometrique(p):
2      '''
3      Renvoie le rang du premier succès dans une suite d'épreuves de Bernoulli
4      indépendantes de paramètre p.
5      '''
6      rang = 1      # numéro de l'épreuve en cours
7      while bernoulli(p) == 0:      # tant que l'épreuve en cours est un échec...
8          rang += 1      # ... on passe à la suivante
9      return rang

```

2. On rappelle que si $X \hookrightarrow \mathcal{G}(p)$, alors $E(X) = \frac{1}{p}$ et $V(X) = \frac{1-p}{p^2}$. On réutilise les fonctions `moyenne` et `variance_empirique`.

Attention ! Ne pas confondre cette question avec la précédente : ici on vérifie l'espérance et la variance, pas la loi.

```

1  N = 10000
2  for p in [0.2, 0.5, 0.8]:

```

```

3     echantillon = [variable_geometrique(p) for k in range(N)]
4     print("p =", p)
5     print("    moyenne empirique :", moyenne(echantillon), "; espérance :", 1 / p)
6     print("    variance empirique :", variance_empirique(echantillon),
7           "; variance :", (1 - p) / p**2)

```

Pour $p = 0,2$, on obtient par exemple :

```

p = 0.2
moyenne empirique : 5.0233 ; espérance : 5.0
variance empirique : 19.52555710999855 ; variance : 19.999999999999996

```

3. a) On compte d'abord les effectifs de chaque valeur obtenue dans un dictionnaire, puis on fabrique le dictionnaire des fréquences, en rangeant les clés dans l'ordre croissant.

```

1     def estime_loi(p, nb_essais=10000):
2         effectifs = {}      # effectifs[x] : nombre de simulations ayant donné
3         for k in range(nb_essais):
4             x = variable_geometrique(p)
5             if x in effectifs:
6                 effectifs[x] += 1
7             else:
8                 effectifs[x] = 1    # première apparition de la valeur x
9         loi = {}            # loi[x] : fréquence de la valeur x
10        for x in sorted(effectifs):    # sorted(effectifs) : liste triée des clés
11            loi[x] = effectifs[x] / nb_essais
12        return loi

```

Depuis la version 3.7 de Python, un dictionnaire se parcourt dans l'ordre où les clés ont été insérées : les clés de `loi` sont donc parcourues dans l'ordre croissant, ce qui est plus agréable pour l'affichage et pour le tracé.

Vérification, en se rappelant que $P(X = k) = p(1 - p)^{k-1}$ pour $k \in \mathbb{N}^*$:

```

1     p = 0.3
2     loi = estime_loi(p)
3     for x in loi:
4         print(x, ": fréquence", loi[x], "; probabilité", p * (1 - p)**(x - 1))

```

```

1 : fréquence 0.3062 ; probabilité 0.3
2 : fréquence 0.2109 ; probabilité 0.21
3 : fréquence 0.1502 ; probabilité 0.14699999999999996
4 : fréquence 0.1029 ; probabilité 0.10289999999999998
...

```

b)

```

1     import matplotlib.pyplot as plt
2
3     def trace_loi_geometrique(p, nb_essais=10000):
4         loi = estime_loi(p, nb_essais)
5         valeurs = [x for x in loi]          # valeurs obtenues, dans l'ordre croissant
6         frequences = [loi[x] for x in valeurs] # fréquences observées
7         probas = [p * (1 - p)**(x - 1) for x in valeurs] # probabilités théoriques
8         plt.bar(valeurs, frequences, width=0.5, color='lightgray', edgecolor='black',

```

```

9         label='fréquences observées')
10     plt.plot(valeurs, probas, color='red', linestyle='--', marker='o',
11              label='probabilités théoriques')
12     plt.legend()
13     plt.show()

```

Pourquoi pas `plt.hist`? On peut être tenté d'utiliser `plt.hist(echantillon, density=True, bins=max(echantillon))`, qui découpe l'intervalle $[1, M]$ (où M est la plus grande valeur obtenue) en M classes de même largeur. Mais ces classes ont alors pour largeur $\frac{M-1}{M}$ et non 1 : avec `density=True`, les hauteurs des barres sont les fréquences divisées par cette largeur, donc multipliées par $\frac{M}{M-1}$. Le résultat est faux, très légèrement (on ne le voit guère quand M est grand, d'où le piège...). Si l'on tient à `plt.hist`, il faut imposer des classes de largeur 1 centrées sur les entiers : `bins=[k - 0.5 for k in range(1, M + 2)]`. Le diagramme en bâtons construit à partir du dictionnaire est plus simple et plus sûr.

4. a) 1

```
def repartition_empirique(liste, t):
```


2

```
    compteur = 0          # nombre d'éléments de liste inférieurs ou égaux à
```


3

```
    for x in liste:
```


4

```
        if x <= t:
```


5

```
            compteur += 1
```


6

```
    return compteur / len(liste)
```

b) Soit $k \in \mathbb{N}$. L'événement $(X > k)$ est réalisé si et seulement si les k premières épreuves sont des échecs. Par **indépendance** des épreuves, $P(X > k) = (1 - p)^k$, donc

$$\forall k \in \mathbb{N}, F_X(k) = 1 - (1 - p)^k$$

c) 1

```
def trace_repartition_geometrique(p, nb_essais=10000, k_max=15):
```


2

```
    echantillon = [variable_geometrique(p) for k in range(nb_essais)]
```


3

```
    abscisses = [k for k in range(k_max + 1)]
```


4

```
    F_emp = [repartition_empirique(echantillon, k) for k in abscisses] # empirique
```


5

```
    F_th = [1 - (1 - p)**k for k in abscisses] # théorique
```


6

```
    plt.step(abscisses, F_emp, where='post', color='blue', linestyle='--',
```


7

```
             label='fonction de répartition empirique')
```


8

```
    plt.plot(abscisses, F_th, color='red', linestyle='--', marker='o',
```


9

```
             label='valeurs théoriques')
```


10

```
    plt.legend()
```


11

```
    plt.show()
```

5. **Idée :** la probabilité conditionnelle $P_{(X>n)}(X > n+k)$ s'estime par la fréquence de l'événement $(X > n+k)$ parmi les simulations où $(X > n)$ est réalisé.

```

1 def absence_memoire(p, n, k, nb_essais=100000):
2     nb_sup_n = 0          # nombre de simulations où X > n
3     nb_sup_n_plus_k = 0   # nombre de simulations où X > n + k (donc aussi X > n)
4     for i in range(nb_essais):
5         x = variable_geometrique(p)
6         if x > n:
7             nb_sup_n += 1
8             if x > n + k:

```

```

9         nb_sup_n_plus_k += 1
10    return nb_sup_n_plus_k / nb_sup_n

```

```

>>> absence_memoire(0.3, 4, 2), 0.7**2
(0.4909106036024793, 0.48999999999999994)

```

On trouve bien $P_{(X>n)}(X > n + k) \approx P(X > k) = (1 - p)^k$: c'est la propriété d'*absence de mémoire* de la loi géométrique. Savoir qu'on a déjà subi n échecs ne change rien à la loi du nombre d'épreuves qu'il reste à attendre.

Version 2, avec numpy. Il n'y a pas de façon simple de vectoriser la boucle `while` de `variable_geometrique`, mais une fois l'échantillon converti en tableau `numpy`, les fonctions `np.mean` et `np.var` calculent directement la moyenne et la variance empiriques :

```

1  echantillon = np.array([variable_geometrique(0.2) for k in range(10000)])
2  print(np.mean(echantillon), np.var(echantillon))

```

Enfin, `np.random.geometric(p)` simule directement une variable de loi $\mathcal{G}(p)$, et `np.random.geometric(p, N)` renvoie un tableau de N simulations.

Correction de l'Exercice 11 *Tirages sans remise*

La loi de X s'appelle la *loi hypergéométrique*. Elle ne fait pas partie des lois à connaître dans le cours, mais elle est une bonne source d'exercices. Si vous ne vous souvenez pas bien de ce type de situation, commencez par revoir le poly *Lois classiques*.

On peut la paramétrer de deux manières :

- par (a, b, n) , comme ici : nombre initial de boules blanches, nombre initial de boules noires, nombre de boules tirées ;
- ou par (N, n, p) : nombre total de boules, nombre de boules tirées, proportion initiale de boules blanches.

Le lien entre les deux écritures est $N = a + b$ et $p = \frac{a}{N}$.

1. a) **Idée** : à chaque tirage, s'il reste a boules blanches et b boules noires, la probabilité de tirer une blanche est $\frac{a}{a+b}$. On simule donc l'événement « tirer une blanche » par l'événement « `random() < a/(a+b)` », qui a la même probabilité, puis on met à jour a ou b .

```

1  def tirages_sans_remise_1(a, b, n):
2      '''
3      a, b : nombres initiaux de boules blanches et noires ; n : nombre de tirages
4      Renvoie le nombre de boules blanches tirées.
5      '''
6      nb_blanches = 0          # nombre de boules blanches tirées jusqu'ici
7      for k in range(n):
8          if random() < a / (a + b): # on tire une blanche
9              nb_blanches += 1
10             a -= 1              # a : nombre de blanches restant dans l'urne
11             else:              # on tire une noire
12                 b -= 1         # b : nombre de noires restant dans l'urne
13     return nb_blanches

```

- b) On représente l'urne par une liste contenant a fois le nombre 1 (les blanches) et b fois le nombre 0 (les noires). On tire une boule au hasard en choisissant un indice au hasard, et on la retire de l'urne avec la méthode `.pop(i)`.

Rappel : la commande `maliste.pop(i)` fait DEUX choses : elle enlève à `maliste` l'élément d'indice i , ET ELLE LE RENVOIE (il peut donc être utilisé dans un calcul, une affectation, etc.).

```

1  def tirages_sans_remise_2(a, b, n):
2      urne = [1] * a + [0] * b      # 1 pour une boule blanche, 0 pour une noire
3      nb_blanches = 0                # nombre de boules blanches tirées jusqu'ici
4      for k in range(n):
5          i = randint(0, len(urne) - 1)  # indice de la boule tirée
6          nb_blanches += urne.pop(i)      # on retire la boule ; +1 si elle est blanche
7      return nb_blanches

```

La fonction `choice` n'est pas un bon choix ici (ha ha) : elle renvoie un élément de la liste, mais ne l'enlève pas de l'urne. On aurait alors simulé des tirages *avec* remise.

2. On réutilise les fonctions `k_parmi_n`, `frequence` et `moyenne`.

```

1  def proba_hypergeometrique(a, b, n, k):
2      return k_parmi_n(k, a) * k_parmi_n(n - k, b) / k_parmi_n(n, a + b)
3
4  a, b, n = 5, 8, 6
5  N = 10000
6  echantillon_1 = [tirages_sans_remise_1(a, b, n) for i in range(N)]
7  echantillon_2 = [tirages_sans_remise_2(a, b, n) for i in range(N)]
8  for k in range(n + 1):
9      print("k =", k, ":", frequence(echantillon_1, k), frequence(echantillon_2, k),
10          proba_hypergeometrique(a, b, n, k))
11  print("moyenne empirique :", moyenne(echantillon_1), "; espérance :", n * a / (a + b))

```

```

k = 0 : 0.0163 0.0176 0.016317016317016316
k = 1 : 0.1683 0.1592 0.16317016317016317
k = 2 : 0.4033 0.4104 0.40792540792540793
k = 3 : 0.3298 0.325 0.32634032634032634
k = 4 : 0.0778 0.0828 0.08158508158508158
k = 5 : 0.0045 0.005 0.004662004662004662
k = 6 : 0.0 0.0 0.0
moyenne empirique : 2.298 ; espérance : 2.3076923076923075

```

Correction de l'Exercice 12 *Simuler une variable aléatoire finie quelconque*

1. • Représentation des données :

La loi de X est donnée par la liste `liste_probab` des $n + 1$ valeurs $p_0, p_1, \dots, p_n$, positives et de somme 1.

- But :

Écrire une fonction `simule_X(liste_probab)` qui renvoie un entier k compris entre 0 et n (où $n + 1$ est la longueur de `liste_probab`), de telle sorte que la probabilité de renvoyer k soit exactement p_k .

- Principe :

- a) On découpe $[0, 1[$ en $n + 1$ intervalles $I_0, \dots, I_n$ deux à deux disjoints, tels que pour tout k , la longueur de I_k soit exactement p_k . Pour cela, on pose, pour tout $k \in \llbracket 0; n \rrbracket$,

$$S_k = p_0 + p_1 + \dots + p_k = F_X(k),$$

de sorte que $S_k - S_{k-1} = p_k$ pour $k \geq 1$, et

$$I_0 = [0, S_0[\text{ et } \forall k \in \llbracket 1; n \rrbracket, I_k = [S_{k-1}, S_k[.$$

Comme $S_n = p_0 + \dots + p_n = 1$, ces intervalles recouvrent bien $[0, 1[$.

- b) On pose $u = \text{random}()$. Il existe un unique k tel que $u \in I_k$, et d'après la propriété fondamentale de $\text{random}()$, la probabilité que u tombe dans I_k est la longueur de I_k , c'est-à-dire p_k .
- c) Il reste à renvoyer cet indice k : il a bien une probabilité p_k d'être renvoyé, ce qu'on voulait depuis le début.
- d) Mais comment déterminer ce k ? C'est le premier indice tel que $u < S_k$. On calcule donc les S_i successifs, et on avance tant que $S_i \leq u$. Dès que $S_i > u$, on a trouvé : $k = i$.

```

1  def simule_X(liste_probab):
2      '''
3      Renvoie un entier k entre 0 et n = len(liste_probab) - 1,
4      avec probabilité liste_probab[k].
5      '''
6      u = random()
7      i = 0                # indice de l'intervalle testé
8      S = liste_probab[0]  # contient S_i = p_0 + ... + p_i, c'est-à-dire F_X(
9      while S <= u:        # u n'est pas dans I_i : on passe à l'intervalle su
10         i += 1
11         S += liste_probab[i]
12     return i             # u est dans I_i

```

Un problème caché. Cette version est correcte mathématiquement, mais pas tout à fait informatiquement : les flottants sont calculés de façon approchée, et la somme des `liste_probab[i]` peut valoir un tout petit peu moins que 1 (voir la question 4). Si `random()` renvoie un nombre très proche de 1, la boucle peut alors dépasser le dernier indice de la liste, et l'on obtient une erreur `IndexError`. C'est extrêmement rare... mais « extrêmement rare », pour un programme lancé des millions de fois, finit par arriver.

Cette idée se généralise à toute variable dont on connaît la fonction de répartition : variables discrètes infinies (voir l'exercice suivant), et même variables à densité, comme on le verra plus tard.

```

2. 1  N = 10000
2     probas_exemple = [0.1, 0.6, 0.15, 0.15]    # c'est bien une loi : la somme vaut 1
3     echantillon = [simule_X(probas_exemple) for k in range(N)]
4     for valeur in range(len(probas_exemple)):
5         print("fréquence de", valeur, ":", frequence(echantillon, valeur))

```

```

fréquence de 0 : 0.0962
fréquence de 1 : 0.6125
fréquence de 2 : 0.1451
fréquence de 3 : 0.1462

```

3. On veut simuler une variable X prenant $n + 1$ valeurs arbitraires (mais connues) $v_0, \dots, v_n$ avec des probabilités $p_0, \dots, p_n$. La fonction prend donc en paramètres deux listes de même taille, **valeurs** et **probas**, qui contiennent respectivement $[v_0, \dots, v_n]$ et $[p_0, \dots, p_n]$, avec $p_k = P(X = v_k)$.

Principe : on se ramène tout simplement au cas précédent ! On tire au hasard un indice entre 0 et n suivant la loi donnée par **probas**, puis on renvoie l'élément de **valeurs** correspondant à cet indice.

```
1 def simule_X_quelconque(valeurs, probas):
2     return valeurs[simule_X(probas)]
```

```
>>> fruits = ["Pomme", "Fraise", "Tomate"]
>>> [simule_X_quelconque(fruits, [0.5, 0.3, 0.2]) for k in range(6)]
['Pomme', 'Pomme', 'Tomate', 'Pomme', 'Fraise', 'Pomme']
```

Notons que `np.random.choice(valeurs, p = probas)` fait ce travail.

4. Le script affiche `0.9999999999999999 True` : à cause des arrondis, la somme de dix flottants 0.1 est strictement inférieure à 1.

C'est exactement le calcul fait par **simule_X**. Avec `liste_probab = [0.1] * 10`, si `random()` renvoie un nombre $u \geq 0,9999999999999999$, toutes les sommes S_i sont inférieures ou égales à u : la boucle tente d'accéder à `liste_probab[10]`, qui n'existe pas.

Curiosité : dans les versions récentes de Python, `sum([0.1] * 10)` renvoie bien 1.0, car la fonction `sum` y utilise un algorithme de sommation plus précis. Mais l'accumulation pas à pas faite dans **simule_X** n'en profite pas.

Version protégée : on interdit à l'indice de dépasser n . Si l'on arrive au dernier intervalle, c'est forcément le bon (aux erreurs d'arrondi près).

```
1 def simule_X(liste_probab):
2     n = len(liste_probab) - 1 # plus grande valeur possible
3     u = random()
4     i = 0
5     S = liste_probab[0]
6     while S <= u and i < n:    # garde-fou : on ne dépasse jamais l'indice n
7         i += 1
8         S += liste_probab[i]
9     return i
```

Correction de l'Exercice 13 *Simuler une loi de Poisson*

1. **Idée :** comme dans l'exercice précédent, on connaît la fonction de répartition de X :

$$\forall k \in \mathbb{N}, F_X(k) = \sum_{i=0}^k e^{-a} \frac{a^i}{i!}.$$

On renvoie le premier entier k tel que $F_X(k) > u$, où $u = \text{random}()$. La seule différence est que $X(\Omega) = \mathbb{N}$ est infini : on ne peut pas stocker la loi dans une liste, on calcule ses termes au fur et à mesure.

```

a) 1  from math import exp, factorial
    2
    3  def simule_poisson_naif(a):
    4      u = random()
    5      k = 0          # valeur testée
    6      F = exp(-a)    # contient F_X(k) ; initialisée à F_X(0) = P(X = 0)
    7      while F <= u:
    8          k += 1
    9          F += exp(-a) * a**k / factorial(k)    # on ajoute P(X = k)
   10      return k

```

- b) **Idée** : pour tout $k \geq 1$, $P(X = k) = e^{-a} \frac{a^k}{k!} = \frac{a}{k} \times e^{-a} \frac{a^{k-1}}{(k-1)!} = \frac{a}{k} P(X = k-1)$. On peut donc mettre à jour $P(X = k)$ à partir de $P(X = k-1)$ par une seule multiplication.

```

1  def simule_poisson(a):
2      u = random()
3      k = 0          # valeur testée
4      terme = exp(-a) # contient P(X = k) ; initialisé à P(X = 0)
5      F = terme      # contient F_X(k)
6      while F <= u:
7          k += 1
8          terme *= a / k # passage de P(X = k-1) à P(X = k)
9          F += terme
10     return k

```

La version naïve recalcule à chaque tour de boucle une puissance et une factorielle, ce qui demande de l'ordre de k multiplications au tour k . La seconde n'en fait qu'une par tour : c'est beaucoup plus économique, et c'est une astuce très fréquente dès qu'on manipule des sommes de termes $\frac{a^k}{k!}$ (voir aussi la fonction `proba_exacte` de l'exercice sur le tirage au sort des cadeaux).

2. On rappelle que $E(X) = V(X) = a$.

```

1  def proba_poisson(a, k):
2      return exp(-a) * a**k / factorial(k)
3
4  a = 3
5  N = 10000
6  echantillon_1 = [simule_poisson_naif(a) for k in range(N)]
7  echantillon_2 = [simule_poisson(a) for k in range(N)]
8  for k in range(10):
9      print("k =", k, ":", frequence(echantillon_1, k), frequence(echantillon_2,
10         proba_poisson(a, k))
11  print("moyenne empirique :", moyenne(echantillon_2),
12        "; variance empirique :", variance_empirique(echantillon_2))

```

```

k = 0 : 0.049 0.0487 0.049787068367863944
k = 1 : 0.1519 0.156 0.14936120510359183
k = 2 : 0.2241 0.2235 0.22404180765538775
k = 3 : 0.2268 0.2204 0.22404180765538775
k = 4 : 0.1665 0.1712 0.16803135574154082

```

```
...
moyenne empirique : 2.9839 ; variance empirique : 3.0134407899999935
```

```
3. 1 def simule_poisson_binomiale(a, n=1000):
2     return variable_binomiale(n, a / n)
3
4     echantillon_3 = [simule_poisson_binomiale(3) for k in range(N)]
5     for k in range(8):
6         print("k =", k, ":", frequence(echantillon_3, k), proba_poisson(3, k))
```

On obtient, pour k de 0 à 7, les fréquences 0,0492 ; 0,1424 ; 0,2324 ; 0,2233 ; 0,1636 ; 0,1031 ; 0,0502 ; 0,0234 : elles sont tout à fait compatibles avec la loi $\mathcal{P}(3)$.

Justification théorique

Pour tout $k \in \mathbb{N}$, $\binom{n}{k} \left(\frac{a}{n}\right)^k \left(1 - \frac{a}{n}\right)^{n-k} \xrightarrow{n \rightarrow +\infty} e^{-a} \frac{a^k}{k!}$. On dira plus tard en maths que la suite des lois binomiales $\mathcal{B}\left(n, \frac{a}{n}\right)$ converge en loi vers la loi de Poisson $\mathcal{P}(a)$ (chapitre sur les théorèmes limites), et l'on précisera alors les critères usuels qui permettent d'approcher une loi binomiale par une loi de Poisson. C'est aussi ce qui explique que la loi de Poisson soit appelée la « loi des événements rares » : le nombre de succès parmi un grand nombre n d'épreuves indépendantes, chacune de petite probabilité de succès, suit approximativement une loi de Poisson.

Cette méthode est simple, mais coûteuse : chaque simulation demande $n = 1000$ appels à `random()`, contre quelques-uns seulement pour `simule_poisson`. Et ce n'est qu'une approximation. Notons enfin que `np.random.poisson(a)` simule directement une variable de loi $\mathcal{P}(a)$.

Correction du Problème 14 *Chi-Fou-Mi*

1. La liste `attaques` permet de choisir une attaque au hasard avec `choice(attaques)`. Le dictionnaire `victime` donne, pour chaque attaque, l'attaque qu'elle bat : `victime["Papier"]` vaut "Caillou". Cela évite d'écrire une cascade de `if` pour déterminer le vainqueur d'une manche.
2. Voir la proposition de structure de l'énoncé.

```
3. 1 from random import choice
2
3     attaques = ["Papier", "Caillou", "Ciseau"]
4     # victime[x] est l'attaque battue par l'attaque x
5     victime = {"Papier": "Caillou", "Caillou": "Ciseau", "Ciseau": "Papier"}
6
7     def resultat(attaque_0, attaque_1):
8         '''
9         Renvoie -1 si la manche est nulle (attaques égales),
10        et sinon le numéro (0 ou 1) de l'attaque gagnante.
11        '''
12        if attaque_0 == attaque_1:
13            return -1
14        elif victime[attaque_0] == attaque_1: # attaque_0 bat attaque_1
15            return 0
16        else:
17            return 1
```

Les deux adversaires sont numérotés 0 et 1, ce qui simplifie le stockage des informations les concernant (noms, scores) : la fonction `resultat` renvoie soit `-1` (manche nulle), soit directement l'indice de l'adversaire qui gagne la manche, qui sert alors à mettre à jour la liste des scores.

```

1  def chi_fou_mi(m=2, affichage=False, joueurs=["Ariane", "Boualem"]):
2      '''
3      Simule une partie à écart m entre deux adversaires jouant au hasard,
4      et renvoie la durée de la partie (en manches).
5      - affichage : True si l'on veut la description de la partie
6      - joueurs : liste des noms des deux adversaires
7      '''
8      scores = [0, 0]      # scores[i] : nombre de manches gagnées par l'adversair i
9      nb_manches = 0      # nombre de manches jouées
10     # la partie continue tant que l'écart des scores (en valeur absolue) n'a pa atteint m
11     while abs(scores[0] - scores[1]) < m:
12         nb_manches += 1
13         attaque_0 = choice(attaques)      # attaque de l'adversaire 0
14         attaque_1 = choice(attaques)      # attaque de l'adversaire 1
15         gagnant = resultat(attaque_0, attaque_1)      # -1, 0 ou 1
16         if gagnant != -1:                  # s'il n'y a pas manche nulle...
17             scores[gagnant] += 1
18         # la suite ne concerne que l'affichage (si celui-ci est demandé)
19         if affichage:
20             print("manche", nb_manches)
21             print(joueurs[0], ":", attaque_0, "!!")
22             print(joueurs[1], ":", attaque_1, "!!")
23             if gagnant == -1:
24                 print("manche nulle")
25             else:
26                 print("un point pour", joueurs[gagnant])
27             print("scores :")
28             for i in range(2):
29                 print(joueurs[i], ":", scores[i])
30             print("-----")
31         if affichage:
32             if scores[0] > scores[1]:
33                 print("Le vainqueur est :", joueurs[0])
34             else:
35                 print("Le vainqueur est :", joueurs[1])
36     return nb_manches

```

```

1  def estime_loi_chifoumi(m=2, nb_essais=100000):
2      effectifs = {}      # effectifs[s] : nombre de parties ayant duré s manches
3      for k in range(nb_essais):
4          s = chi_fou_mi(m)
5          if s in effectifs:
6              effectifs[s] += 1
7          else:
8              effectifs[s] = 1
9      loi = {}            # loi[s] : fréquence des parties ayant duré s manches
10     for s in sorted(effectifs):
11         loi[s] = effectifs[s] / nb_essais
12     return loi
13
14 def estime_esperance_chifoumi(m=2, nb_essais=100000):
15     somme = 0            # somme des durées des parties simulées

```

```

16     for k in range(nb_essais):
17         somme += chi_fou_mi(m)
18     return somme / nb_essais

```

4. a) Pour tout $i \in \mathbb{N}^*$, notons N_i l'événement « la i -ème manche est nulle ». Les deux adversaires choisissant leurs attaques indépendamment et de façon équiprobable, chacun des 9 couples d'attaques possibles a pour probabilité $\frac{1}{9}$, et 3 d'entre eux donnent une manche nulle : $P(N_i) = \frac{1}{3}$. Les manches étant indépendantes, les événements N_i sont mutuellement indépendants.

Pour $m = 1$, la partie s'arrête dès la première manche non nulle. Ainsi, S_1 est le rang du premier succès dans une suite d'épreuves de Bernoulli indépendantes de même paramètre $\frac{2}{3}$ (où un succès est une manche non nulle) :

$$S_1 \hookrightarrow \mathcal{G}\left(\frac{2}{3}\right) \text{ et } E(S_1) = \frac{3}{2}$$

Rédaction bis, sans invoquer le modèle. Pour tout $n \in \mathbb{N}^*$, $(S_1 = n) = N_1 \cap \dots \cap N_{n-1} \cap \overline{N_n}$, donc par **indépendance** des manches :

$$P(S_1 = n) = P(N_1) \times \dots \times P(N_{n-1}) \times P(\overline{N_n}) = \left(\frac{1}{3}\right)^{n-1} \frac{2}{3}.$$

```

b) 1     loi = estime_loi_chifoumi(1)
2     for s in loi:
3         print(s, ": fréquence", loi[s], "; probabilité", (2/3) * (1/3)**(s - 1)
4         print("moyenne empirique :", estime_esperance_chifoumi(1), "; espérance :", 3/2)

```

```

1 : fréquence 0.66463 ; probabilité 0.6666666666666666
2 : fréquence 0.22316 ; probabilité 0.2222222222222222
3 : fréquence 0.07487 ; probabilité 0.07407407407407407
4 : fréquence 0.02496 ; probabilité 0.02469135802469135
...
moyenne empirique : 1.50261 ; espérance : 1.5

```

```

5. a) 1     import matplotlib.pyplot as plt
2
3     def trace_esperances(m_max=8, nb_essais=10000):
4         liste_m = [m for m in range(1, m_max + 1)]
5         moyennes = [estime_esperance_chifoumi(m, nb_essais) for m in liste_m]
6         quotients = [moyennes[i] / liste_m[i]**2 for i in range(m_max)]
7         print("moyennes empiriques :", moyennes)
8         print("quotients par m^2 :", quotients)
9         plt.plot(liste_m, moyennes, color='blue', linestyle='-', marker='o',
10                  label='moyennes empiriques')
11         plt.legend()
12         plt.show()

```

Pour m de 1 à 6, on obtient par exemple les moyennes empiriques 1,50 ; 6,00 ; 13,72 ; 24,01 ; 37,58 ; 53,49, et des quotients par m^2 tous proches de 1,5.

b) On conjecture que :

$$E(S_m) = \frac{3m^2}{2}$$

On peut le démontrer : l'écart des scores effectue ce qu'on appelle une *marche aléatoire*, et la question revient à un problème célèbre, celui de la *ruine du joueur*. C'est hors de portée de ce TP, mais on peut au moins comprendre le facteur $\frac{3}{2}$: en moyenne, une manche sur trois est nulle, donc il faut $\frac{3}{2}$ fois plus de manches que si les manches nulles n'existaient pas.

Le niveau 3 confirme la conjecture pour $m = 2$.

6. a) Avant toute manche, la partie n'est pas terminée et les scores sont égaux : $a_0 = 1$ et $b_0 = 0$. À l'issue de la première manche, la partie n'est pas terminée (puisque $m = 2$), et les scores sont égaux si et seulement si la manche est nulle : $a_1 = \frac{1}{3}$ et $b_1 = \frac{2}{3}$.
- b) Soit $n \in \mathbb{N}$. Pour $j \in \{0, 1\}$, notons G_{n+1}^j l'événement « l'adversaire j gagne la manche $n + 1$ » et U_n^j l'événement « à l'issue de n manches, la partie n'est pas terminée et l'adversaire j a un point d'avance », de sorte que U_n est la réunion disjointe de U_n^0 et U_n^1 . Comme pour N_{n+1} , $P(G_{n+1}^j) = \frac{3}{9} = \frac{1}{3}$.

• Calcul de a_{n+1} :

À l'issue de $n + 1$ manches, la partie n'est pas terminée et les scores sont égaux si et seulement si :

- SOIT, à l'issue de n manches, la partie n'était pas terminée avec des scores égaux, et la manche $n + 1$ est nulle ;
- SOIT, à l'issue de n manches, la partie n'était pas terminée et l'un des adversaires avait un point d'avance, et l'autre gagne la manche $n + 1$.

Autrement dit :

$$Z_{n+1} = (Z_n \cap N_{n+1}) \cup (U_n^0 \cap G_{n+1}^1) \cup (U_n^1 \cap G_{n+1}^0),$$

et cette réunion est **disjointe**. Les événements Z_n , U_n^0 et U_n^1 ne dépendent que des n premières manches, qui sont **indépendantes** de la manche $n + 1$ (lemme des coalitions). Donc :

$$\begin{aligned} a_{n+1} &= P(Z_n)P(N_{n+1}) + P(U_n^0)P(G_{n+1}^1) + P(U_n^1)P(G_{n+1}^0) \\ &= \frac{1}{3}a_n + \frac{1}{3}(P(U_n^0) + P(U_n^1)), \end{aligned}$$

d'où

$$a_{n+1} = \frac{a_n + b_n}{3}$$

• Calcul de b_{n+1} :

De même, $U_{n+1} = (Z_n \cap \overline{N_{n+1}}) \cup (U_n \cap N_{n+1})$: soit les scores étaient égaux et la manche n'est pas nulle, soit un adversaire avait un point d'avance et la manche est nulle (si la manche n'est pas nulle, l'écart passe à 0 ou à 2). La réunion est disjointe et, par indépendance :

$$b_{n+1} = \frac{2}{3}a_n + \frac{1}{3}b_n = \frac{2a_n + b_n}{3}$$

c) Soit $n \in \mathbb{N}$. D'après la question précédente, appliquée aux rangs $n+1$ puis n :

$$\begin{aligned}
 b_{n+2} &= \frac{2}{3} a_{n+1} + \frac{1}{3} b_{n+1} \\
 &= \frac{2}{9} (a_n + b_n) + \frac{1}{3} b_{n+1} \\
 &= \frac{1}{3} \left(\frac{2}{3} a_n \right) + \frac{2}{9} b_n + \frac{1}{3} b_{n+1} \\
 &= \frac{1}{3} \left(b_{n+1} - \frac{1}{3} b_n \right) + \frac{2}{9} b_n + \frac{1}{3} b_{n+1} \\
 &= \frac{2}{3} b_{n+1} + \frac{1}{9} b_n,
 \end{aligned}$$

où l'on a utilisé $\frac{2}{3} a_n = b_{n+1} - \frac{1}{3} b_n$ (relation donnant b_{n+1}).

La suite (b_n) est donc récurrente linéaire d'ordre 2, d'équation caractéristique $r^2 - \frac{2}{3}r - \frac{1}{9} = 0$, de discriminant $\frac{4}{9} + \frac{4}{9} = \frac{8}{9} > 0$. Ses racines sont

$$\lambda = \frac{1 + \sqrt{2}}{3} \text{ et } \mu = \frac{1 - \sqrt{2}}{3}.$$

Il existe donc des réels α, β tels que pour tout $n \in \mathbb{N}$, $b_n = \alpha\lambda^n + \beta\mu^n$. La condition $b_0 = 0$ donne $\beta = -\alpha$, puis $b_1 = \alpha(\lambda - \mu) = \alpha \frac{2\sqrt{2}}{3} = \frac{2}{3}$ donne $\alpha = \frac{1}{\sqrt{2}}$. Ainsi :

$$\forall n \in \mathbb{N}, b_n = \frac{1}{\sqrt{2}} \left(\left(\frac{1 + \sqrt{2}}{3} \right)^n - \left(\frac{1 - \sqrt{2}}{3} \right)^n \right)$$

d) Soit $n \in \mathbb{N}^*$. Partant d'une situation où les scores sont égaux, une seule manche ne peut pas terminer la partie. Donc la partie se termine exactement à la manche n si et seulement si, à l'issue de $n-1$ manches, elle n'était pas terminée et l'un des adversaires avait un point d'avance, et cet adversaire gagne la manche n :

$$(S_2 = n) = (U_{n-1}^0 \cap G_n^0) \cup (U_{n-1}^1 \cap G_n^1).$$

La réunion est disjointe et, par indépendance comme précédemment :

$$P(S_2 = n) = \frac{1}{3} (P(U_{n-1}^0) + P(U_{n-1}^1)) = \frac{b_{n-1}}{3}$$

soit, pour tout $n \in \mathbb{N}^*$, $P(S_2 = n) = \frac{1}{3\sqrt{2}} (\lambda^{n-1} - \mu^{n-1})$.

• Somme des probabilités :

On a $\lambda \approx 0,80$ et $\mu \approx -0,14$, donc $|\lambda| < 1$ et $|\mu| < 1$: les séries géométriques $\sum \lambda^{n-1}$ et $\sum \mu^{n-1}$ convergent, et

$$\sum_{n=1}^{+\infty} P(S_2 = n) = \frac{1}{3\sqrt{2}} \left(\frac{1}{1-\lambda} - \frac{1}{1-\mu} \right) = \frac{1}{3\sqrt{2}} \left(\frac{3}{2-\sqrt{2}} - \frac{3}{2+\sqrt{2}} \right) = \frac{1}{\sqrt{2}} \times \frac{2\sqrt{2}}{4-2} = 1.$$

Conclusion : $\sum_{n=1}^{+\infty} P(S_2 = n) = 1$: la partie se termine quasi-certainement.

Rigoureusement, S_2 n'est pas défini sur l'événement « la partie ne se termine jamais ». Mais celui-ci est quasi-impossible, et le programme tolère qu'une variable aléatoire issue d'une expérience aléatoire ne soit pas définie sur un événement de probabilité nulle.

- e) La série $\sum nx^{n-1}$ (série géométrique dérivée) converge pour $|x| < 1$ et a pour somme $\frac{1}{(1-x)^2}$. La série $\sum nP(S_2 = n)$ converge donc, comme combinaison linéaire de deux séries convergentes, et

$$\begin{aligned} E(S_2) &= \frac{1}{3\sqrt{2}} \left(\frac{1}{(1-\lambda)^2} - \frac{1}{(1-\mu)^2} \right) \\ &= \frac{1}{3\sqrt{2}} \left(\frac{9}{(2-\sqrt{2})^2} - \frac{9}{(2+\sqrt{2})^2} \right) \\ &= \frac{3}{\sqrt{2}} \times \frac{(2+\sqrt{2})^2 - (2-\sqrt{2})^2}{((2-\sqrt{2})(2+\sqrt{2}))^2} \\ &= \frac{3}{\sqrt{2}} \times \frac{8\sqrt{2}}{4}. \end{aligned}$$

$$E(S_2) = 6$$

C'est bien la valeur $\frac{3 \times 2^2}{2}$ conjecturée au niveau 2.

```
f) 1  from math import sqrt
    2
    3  lam = (1 + sqrt(2)) / 3      # on évite le nom lambda, mot-clé de Python
    4  mu = (1 - sqrt(2)) / 3
    5
    6  def proba_S2(n):
    7      return (lam**(n - 1) - mu**(n - 1)) / (3 * sqrt(2))
    8
    9  loi = estime_loi_chifoumi(2)
   10  for s in loi:
   11      print(s, ": fréquence", loi[s], "; probabilité", proba_S2(s))
```

```
2 : fréquence 0.22387 ; probabilité 0.2222222222222218
3 : fréquence 0.14725 ; probabilité 0.14814814814814808
4 : fréquence 0.12349 ; probabilité 0.12345679012345674
5 : fréquence 0.09847 ; probabilité 0.09876543209876538
6 : fréquence 0.08009 ; probabilité 0.07956104252400543
...
```

L'exercice « Appelez la SPA ! » de la partie I était en fait une partie à écart 2 sans manche nulle !

Correction de l'Exercice 15 *(**) Le problème des partis*

1. a) On joue des manches tant que personne n'a gagné, en décrémentant le nombre de manches qui manquent au gagnant de chaque manche.

```
1  def fin_de_partie(a, b, p=0.5):
2      '''
3      a, b : nombres de manches manquant à A et à B
4      Renvoie True si A gagne la partie.
```

```

5      '''
6      while a > 0 and b > 0:
7          if random() < p:      # A gagne la manche
8              a -= 1
9          else:                  # B gagne la manche
10             b -= 1
11     return a == 0

```

```

b) 1     def estime_partis(a, b, p=0.5, nb_essais=10000):
2         nb_victoires = 0      # nombre de parties gagnées par A
3         for k in range(nb_essais):
4             if fin_de_partie(a, b, p):
5                 nb_victoires += 1
6         return nb_victoires / nb_essais

```

c) On trouve $\text{estime_partis}(1, 2) \approx 0,75$: A doit recevoir les trois quarts des mises, et B un quart. C'est la réponse de Pascal.

2. a) S'il ne manque aucune manche à A, il a gagné : $\pi(0, b) = 1$. S'il ne manque aucune manche à B, A a perdu : $\pi(a, 0) = 0$.
- b) Soit $a, b \in \mathbb{N}^*$. Notons G l'événement « A gagne la prochaine manche » ; $(G, \overline{G})$ est un système complet d'événements, avec $P(G) = p$ et $P(\overline{G}) = q$. Si G est réalisé, il manque désormais $a - 1$ manches à A et b manches à B, et la suite de la partie est formée de manches indépendantes de la première : sachant G , la probabilité que A gagne est donc $\pi(a - 1, b)$. De même, sachant $\overline{G}$, elle vaut $\pi(a, b - 1)$. D'après la formule des probabilités totales :

$$\pi(a, b) = p \pi(a - 1, b) + q \pi(a, b - 1)$$

3. La récursivité est ici naturelle : pour calculer $\pi(a, b)$, la fonction s'appelle elle-même sur $(a - 1, b)$ et sur $(a, b - 1)$. Les cas de base sont ceux de la question 2a, et ils sont toujours atteints, puisque $a + b$ diminue de 1 à chaque appel.

```

1     def partis(a, b, p=0.5):
2         '''
3         Renvoie la probabilité que A gagne, quand il manque a manches à A et b à B.
4         '''
5         if a == 0:             # A a gagné
6             return 1
7         if b == 0:             # B a gagné
8             return 0
9         return p * partis(a - 1, b, p) + (1 - p) * partis(a, b - 1, p)
10
11     for (a, b, p) in [(1, 2, 0.5), (3, 5, 0.5), (4, 2, 0.6), (5, 5, 0.4)]:
12         print("a =", a, "b =", b, "p =", p, ": estimation", estime_partis(a, b, p),
13               "; valeur exacte", partis(a, b, p))

```

```

a = 1 b = 2 p = 0.5 : estimation 0.7467 ; valeur exacte 0.75
a = 3 b = 5 p = 0.5 : estimation 0.7735 ; valeur exacte 0.7734375
a = 4 b = 2 p = 0.6 : estimation 0.3403 ; valeur exacte 0.33696
a = 5 b = 5 p = 0.4 : estimation 0.2661 ; valeur exacte 0.26656768000000001

```

4. a) L'appel `partis(a, b)` compte pour 1. Si $a = 0$ ou $b = 0$, il ne provoque aucun autre appel. Sinon, il provoque les appels `partis(a - 1, b)` et `partis(a, b - 1)`, avec tous ceux qu'ils provoquent eux-mêmes.

```

1 def nb_appels(a, b):
2     if a == 0 or b == 0:
3         return 1
4     return 1 + nb_appels(a - 1, b) + nb_appels(a, b - 1)

```

- b) On obtient (temps mesurés avec la fonction `time` du module `time`, sur un ordinateur ordinaire) :

n	<code>nb_appels(n, n)</code>	temps de <code>partis(n, n)</code>
5	503	négligeable
10	369 511	environ 0,03 s
12	5 408 311	environ 0,4 s
14	80 233 199	environ 6 s

Le nombre d'appels est multiplié par environ 4 quand n augmente de 1 : le temps de calcul explose. La raison est que les mêmes valeurs sont recalculées un très grand nombre de fois. Par exemple, $\pi(a-1, b-1)$ est calculée une première fois lors du calcul de $\pi(a-1, b)$, et une seconde fois lors du calcul de $\pi(a, b-1)$, et ainsi de suite à tous les niveaux.

On peut montrer que `nb_appels(a, b)` vaut $2 \binom{a+b}{a} - 1$. C'est le même phénomène que pour le calcul récursif naïf des termes de la suite de Fibonacci.

5. On calcule chaque valeur une seule fois, et on la range dans le dictionnaire `memo`, avec pour clé le triplet (a, b, p) . Avant de calculer, on regarde si la valeur est déjà connue.

```

1 def partis_memo(a, b, p=0.5, memo={}):
2     '''
3     Version mémoisée : memo[(a, b, p)] contient pi(a, b) pour le paramètre p,
4     une fois cette valeur calculée.
5     '''
6     if a == 0:
7         return 1
8     if b == 0:
9         return 0
10    if (a, b, p) not in memo: # valeur pas encore calculée
11        memo[(a, b, p)] = p * partis_memo(a - 1, b, p, memo) \
12                        + (1 - p) * partis_memo(a, b - 1, p, memo)
13    return memo[(a, b, p)]

```

L'appel `partis_memo(100, 100)` renvoie 0.5 en quelques millisecondes : il n'y a plus qu'environ 100×100 valeurs à calculer, chacune une seule fois.

Attention au piège ! La valeur par défaut `memo={}` n'est créée qu'une seule fois, lors de la définition de la fonction : le même dictionnaire est donc partagé par tous les appels, y compris des appels successifs lancés depuis la console. C'est ce qui rend la mémoïsation efficace... mais c'est aussi pour cela que `p` doit faire partie de la clé. Avec la clé (a, b) seule, on obtient :

```

>>> partis_memo_faux(3, 5, 0.5)
0.7734375
>>> partis_memo_faux(3, 5, 0.9)      # devrait valoir 0.9998235...
0.7734375

```

le second appel réutilisant la valeur calculée pour $p = 0,5$!

Par ailleurs, la profondeur de récursion vaut $a + b$: pour $a + b$ de l'ordre de 1000, Python s'arrête avec une `RecursionError`.

6. a) On modélise l'expérience par une suite d'au moins $a + b - 1$ manches indépendantes, la partie réelle n'utilisant que les premières. Sur ces $a + b - 1$ manches, A gagne j manches et B en gagne $a + b - 1 - j$.
- SOIT $j \geq a$. Alors B gagne au plus $b - 1$ manches : il n'atteint jamais les b manches qui lui manquent, alors que A atteint ses a manches. C'est donc A qui gagne la partie réelle.
 - SOIT $j \leq a - 1$. Alors B gagne au moins b manches, et A n'atteint jamais ses a manches : c'est B qui gagne la partie réelle.

Conclusion : A gagne la partie si et seulement si il gagne au moins a des $a + b - 1$ manches.

- b) Les $a + b - 1$ manches sont des épreuves de Bernoulli indépendantes, de même paramètre p , où le succès est « A gagne la manche ». La variable Y compte le nombre de succès, donc

$$Y \hookrightarrow \mathcal{B}(a + b - 1, p)$$

D'après la question précédente, l'événement « A gagne la partie » est égal à $(Y \geq a)$, c'est-à-dire à la réunion des événements $(Y = k)$ pour $k \in \llbracket a ; a + b - 1 \rrbracket$. Cette réunion est disjointe, donc :

$$\pi(a, b) = \sum_{k=a}^{a+b-1} P(Y = k) = \sum_{k=a}^{a+b-1} \binom{a+b-1}{k} p^k q^{a+b-1-k}$$

c) 1 `def partis_formule(a, b, p=0.5):`
 2 `n = a + b - 1` # nombre de manches qu'on imagine jouer
 3 `s = 0` # somme des $P(Y = k)$ pour k de a à n
 4 `for k in range(a, n + 1):`
 5 `s += proba_binomiale(n, p, k)`
 6 `return s`

On retrouve les mêmes valeurs qu'avec `partis`, avec un calcul instantané. Par exemple, `partis_formule(1, 2)` vaut 0,75 et `partis_formule(4, 2, 0.6)` vaut 0,33696.

Cette question donne aussi une seconde façon de simuler la fin de partie : `variable_binomiale(a + b - 1, p) >= a` vaut `True` avec probabilité $\pi(a, b)$. Elle est plus rapide à écrire... mais elle s'éloigne de l'expérience décrite, puisqu'on joue systématiquement $a + b - 1$ manches.